

Python for Databases & APIs

Flask, connection to database and CRUD operations

Table of Content

- Connection to SQL database
- Connection to NOSQL database
- REST APIs
- Flask
- GraphQL

Connection to SQL database (SQLite)

Start the connection with a predefined database or create if doesn't exist
And creates a cursor object that allows you to execute SQL commands in the database.

```
import sqlite3

# Connect to SQLite database
conn = sqlite3.connect("example.db") # Creates 'my_database.db' if it doesn't exist
cursor = conn.cursor()

# Create a table
cursor.execute('''
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER
)
''')

# Insert Data
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("Alice", 25))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("Bob", 20))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("celia", 23))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("dafny", 19))
conn.commit()

# Read Data
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())

conn.close()
```

✓ 0.0s

Connection to SQL database (SQLite)

```
import sqlite3

# Connect to SQLite database
conn = sqlite3.connect("example.db") # Creates 'my_database.db' if it doesn't exist
cursor = conn.cursor()

# Create a table
cursor.execute('''
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER
)
''')

# Insert Data
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("Alice", 25))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("Bob", 20))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("celia", 23))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("dafny", 19))
conn.commit()

# Read Data
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())

conn.close()
```

✓ 0.0s

Execute your SQL code
Notice: the typing style

At last commit your changes cause
SQLite does not save changes immediately

Connection to SQL database (SQLite)

```
import sqlite3

# Connect to SQLite database
conn = sqlite3.connect("example.db") # Creates 'my_database.db' if it doesn't exist
cursor = conn.cursor()
# Create a table
cursor.execute('''
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER
)
''')

# Insert Data
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("Alice", 25))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("Bob", 20))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("celia", 23))
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("dafny", 19))
conn.commit()

# Read Data
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())

conn.close()
```

✓ 0.0s

This is because `execute()` **only runs the query** but does not fetch the results.

`Close()` is used to close the connection and if you want to run any code you have to start from `sqlite3.connect()`

Connection to NOSQL database (MongoDB)

```
from pymongo import MongoClient
```

```
client = MongoClient("mongodb://localhost:27017/")  
db = client["example_db"]  
collection = db["users"]
```

Insert Data

```
collection.insert_one({"name": "Alice", "age": 25})
```

Read Data

```
print(list(collection.find()))
```

Update Data

```
collection.update_one({"name": "Alice"}, {"$set": {"age": 26}})
```

Delete Data

```
collection.delete_one({"name": "Alice"})
```

Creates a connection to a MongoDB server running locally. (in this situation args are optional)

Selects a database named example_db and create it if doesn't exist

Selects a collection (similar to a "table" in SQL)

Connection to NOSQL database (MongoDB)

```
from pymongo import MongoClient
```

```
client = MongoClient("mongodb://localhost:27017/")  
db = client["example_db"]  
collection = db["users"]
```

```
# Insert Data
```

```
collection.insert_one({"name": "Alice", "age": 25})
```

```
# Read Data
```

```
print(list(collection.find()))
```

```
# Update Data
```

```
collection.update_one({"name": "Alice"}, {"$set": {"age": 26}})
```

```
# Delete Data
```

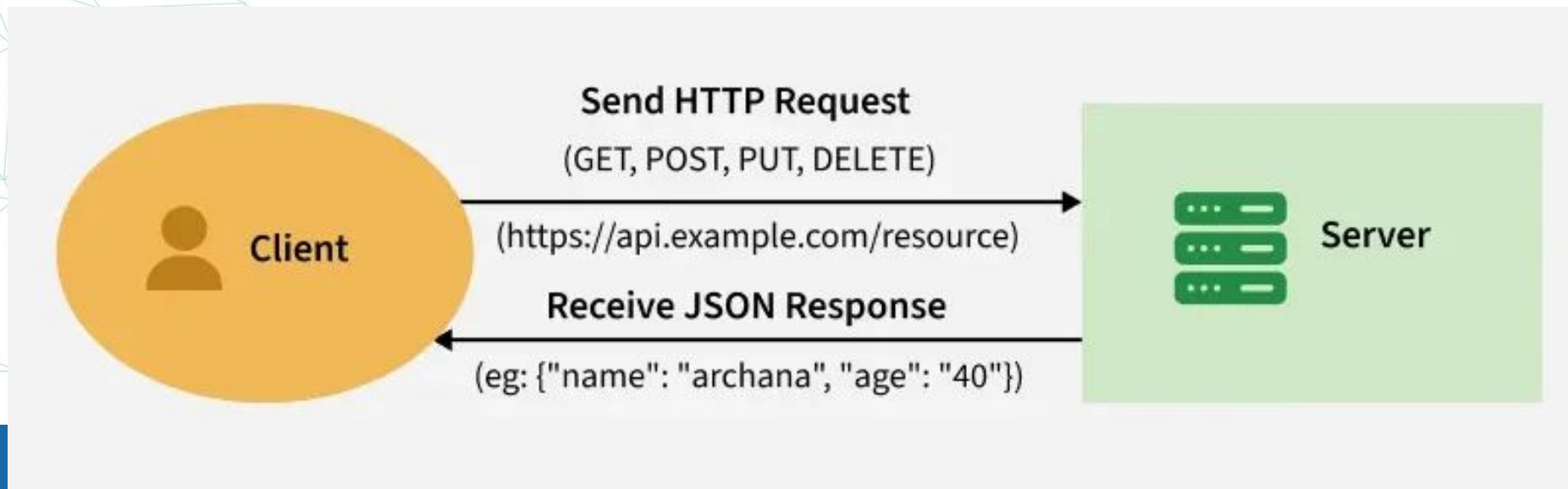
```
collection.delete_one({"name": "Alice"})
```

Operations on NOSQL (insert – select – update – delete)

Python's garbage collector will automatically close connections when the **MongoClient** object is no longer in use, so you don't have to explicitly close the connection

Introduction to REST APIs

- REST API stands for **RE**presentational **S**tate **T**ransfer API.
- allows communication between different systems over the internet.
- work by sending requests and receiving responses, typically in JSON format, between the client and server.



Key Features of REST APIs



Stateless: Each request from a client to a server must contain all the information the server needs to fulfill the request. No session state is stored on the server.



Client-Server Architecture: RESTful APIs are based on a client-server model, where the client and server operate independently, allowing scalability.



Cacheable: Responses from the server can be explicitly marked as cacheable or non-cacheable to improve performance.



Uniform Interface: REST APIs follow a set of conventions and constraints, such as consistent URL paths, standardized HTTP methods, and status codes, to ensure smooth communication.



Layered System: REST APIs can be deployed on multiple layers, which helps with scalability and security.

Various HTTP Methods Used in REST API

HTTP Method	Purpose	Idempotent	Safe	Typical Use Case	Example
GET	Retrieve data (read only)	✓ Yes	✓ Yes	Fetch resources without modifying them	GET /users/123 → returns user data
POST	Create a new resource	✗ No	✗ No	Add new records (users, products, etc.)	POST /users with body { "name": "Ali" }
PUT	Replace a resource entirely	✓ Yes	✗ No	Full update (overwrite existing data)	PUT /users/123 with body { "name": "Ali", "age": 26 }
PATCH	Update part of a resource	✓ Usually	✗ No	Partial update (modify specific fields)	PATCH /users/123 with body { "age": 27 }

📌 Quick Notes:

- **Idempotent** → Running the same request multiple times gives the **same result**.
- **Safe** → Means it doesn't change anything on the server (only **GET** is safe).

Various HTTP Methods Used in REST API

1. GET

The HTTP GET method is used to read (or retrieve) a representation of a resource. In the safe path, GET returns a representation in XML or JSON and an HTTP response code of 200 (OK). In an error case, it most often returns a 404 (NOT FOUND) or 400 (BAD REQUEST).

```
GET /users/123 - Fetches data for user with ID 123.
```

Various HTTP Methods Used in REST API

2. POST

The POST verb is most often utilized to create new resources. In particular, it's used to create subordinate resources. That is, subordinate to some other (e.g. parent) resource. On successful creation, return HTTP status 201, returning a Location header with a link to the newly-created resource with the 201 HTTP status.

```
POST /users with { "name": "anjali", "email": "gfg@example.com" } - Creates a new user.
```

Various HTTP Methods Used in REST API

3. PUT

PUT is an HTTP method used to update or create a resource on the server. When using PUT, the entire resource is sent in the request body, and it replaces the current resource at the specified URL. If the resource doesn't exist, it can create a new one.

```
PUT /users/123 with { "name": "anjali", "email": "gfg@example.com" } - Updates user with ID 123.
```

Various HTTP Methods Used in REST API

4. PATCH

PATCH is an HTTP method used to partially update a resource on the server. Unlike PUT, PATCH only requires the fields that need to be updated to be sent in the request body. It modifies specific parts of the resource rather than replacing the entire resource.

```
PATCH /users/123 with { "email": "new.email@example.com" } - Updates only the email of user 123.
```

Introduction to Web Frameworks in Python

- A web framework helps us build web applications and APIs easily.
- It provides tools to handle requests, send responses, and connect to databases.
- Popular Python Web Frameworks
 - Flask – Small and easy to use. Great for simple APIs.
 - Django – Big and powerful. Best for full websites.
 - FastAPI – Very fast. Good for high-performance APIs.
 - Bottle – Tiny, like Flask, but even smaller.

Flask

- lightweight Python web framework that lets you build APIs and web apps with minimal boilerplate code.
 - **Microframework:** No built-in database/ORM (use SQLite, SQLAlchemy, etc.).
 - **Flexible:** Add only the extensions you need (e.g., Flask-RESTful, Flask-GraphQL).
 - **Development-Friendly:** Built-in debugger and server for rapid prototyping.
 - **Scalable:** Powers apps from small APIs to large-scale systems (Pinterest, LinkedIn use it)



API using flask

```
from flask import Flask, request, jsonify
```

```
app = Flask(__name__)
```

- Create the flask application that could be run at the main

```
users = [{"id": 1, "name": "Alice", "age": 25}]
```

Qodo Gen: Options | Test this function

```
@app.route("/users", methods=["GET"])
```

```
def get_users():
```

```
    return jsonify(users)
```

Qodo Gen: Options | Test this function

```
@app.route("/users", methods=["POST"])
```

```
def add_user():
```

```
    data = request.json
```

```
    users.append({"id": len(users) + 1, "name": data["name"], "age": data["age"]})
```

```
    return jsonify({"message": "User added!"}), 201
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

Python

API using flask

```
from flask import Flask, request, jsonify

app = Flask(__name__)

users = [{"id": 1, "name": "Alice", "age": 25}]
```

Qodo Gen: Options | Test this function

```
@app.route("/users", methods=["GET"])
def get_users():
    return jsonify(users)
```

- A “GET” endpoint to fetch data

Qodo Gen: Options | Test this function

```
@app.route("/users", methods=["POST"])
def add_user():
    data = request.json
    users.append({"id": len(users) + 1, "name": data["name"], "age": data["age"]})
    return jsonify({"message": "User added!"}), 201
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

Python

API using flask

```
from flask import Flask, request, jsonify

app = Flask(__name__)

users = [{"id": 1, "name": "Alice", "age": 25}]
```

Qodo Gen: Options | Test this function

```
@app.route("/users", methods=["GET"])
def get_users():
    return jsonify(users)
```

- A "POST" endpoint to add new record

Qodo Gen: Options | Test this function

```
@app.route("/users", methods=["POST"])
def add_user():
    data = request.json
    users.append({"id": len(users) + 1, "name": data["name"], "age": data["age"]})
    return jsonify({"message": "User added!"}), 201
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

Python

API using flask



```
[  
  - {  
    age: 25,  
    id: 1,  
    name: "Alice"  
  }  
]
```

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on <http://localhost:5000>

Press CTRL+C to quit

127.0.0.1 - - [28/Mar/2025 00:59:14] "GET /users HTTP/1.1" 200 -

Connection to a third-party API

You can connect to APIs from another server using Python:

- GET Request: Fetches posts from a fake API (jsonplaceholder) and prints the first 3.
- POST Request: Sends new data (a post with title, body, userId) to the same API.

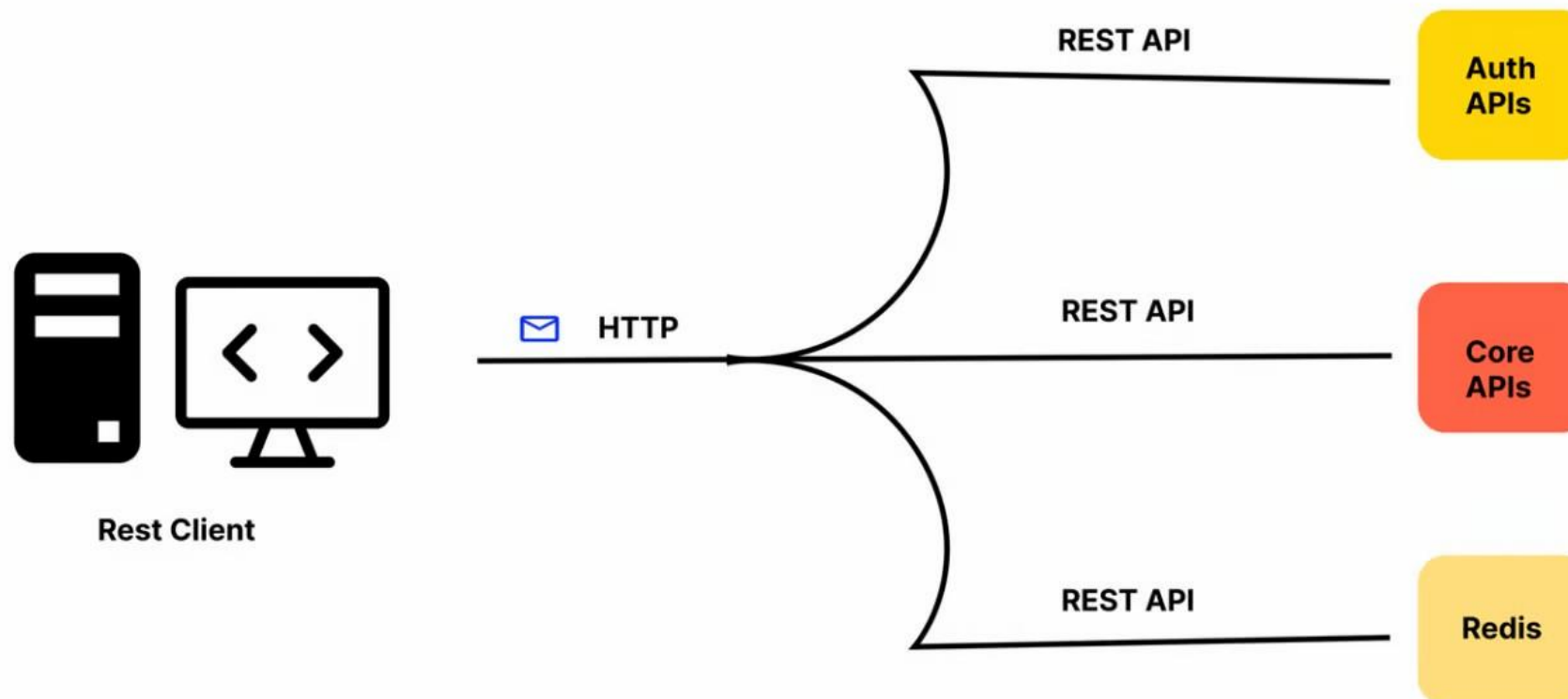
```
import requests

response = requests.get("https://jsonplaceholder.typicode.com/posts")
print(response.json()[:3]) # Print first 3 posts

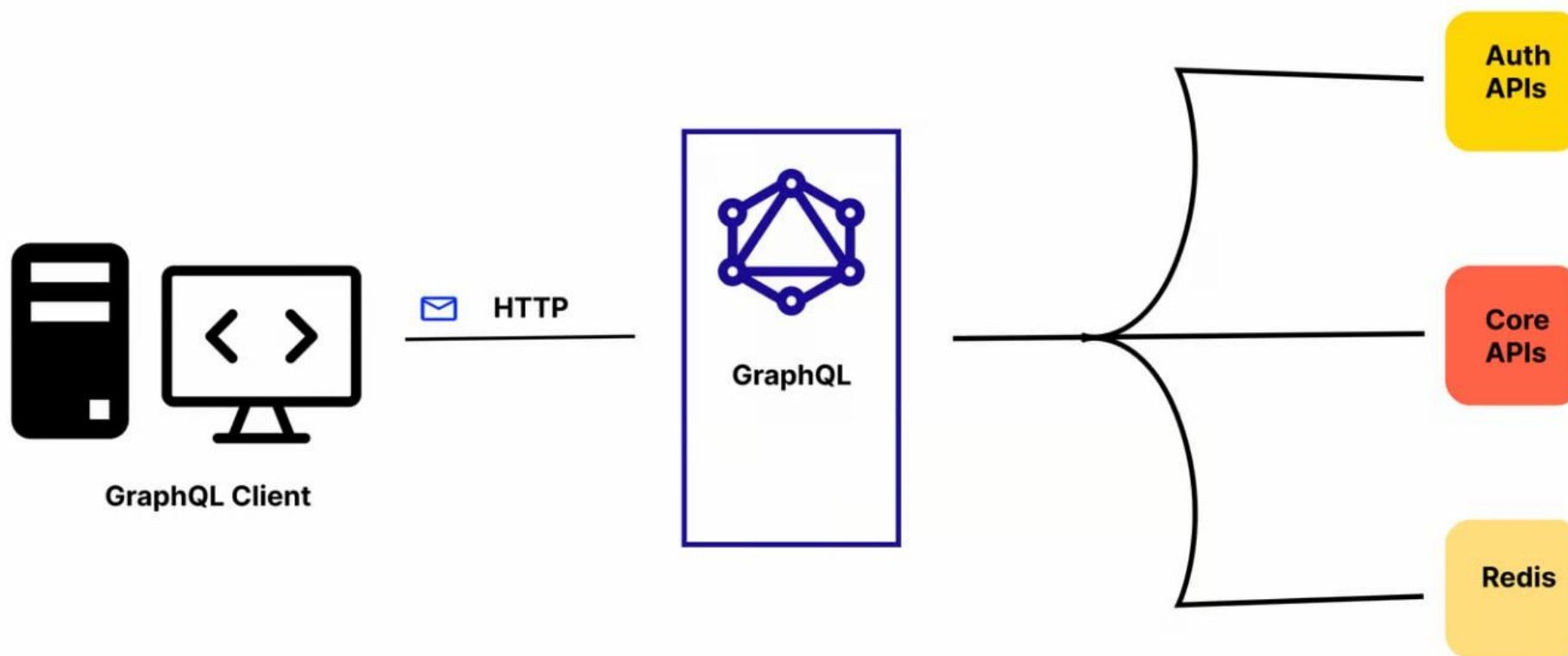
# Sending Data (POST Request)
data = {"title": "New Post", "body": "This is a test.", "userId": 1}
response = requests.post("https://jsonplaceholder.typicode.com/posts", json=data)
print(response.json())
```

✓ 0.9s

REST vs GraphQL



REST vs GraphQL



GraphQL

- While REST APIs require clients to call multiple endpoints for related data (e.g., /users then /users/1/posts), GraphQL lets clients request exactly what they need in a single query.
- Advantages:
 - Single Request
 - No Over-fetching
 - Strong Typing

```
query = """
{
  user(id: 1) {
    name
    posts {
      title
    }
  }
}
"""

response = requests.post("/graphql", json={'query': query})
# {"data": {"user": {"name": "Alice", "posts": [{"title": "Post 1"}]}}
```

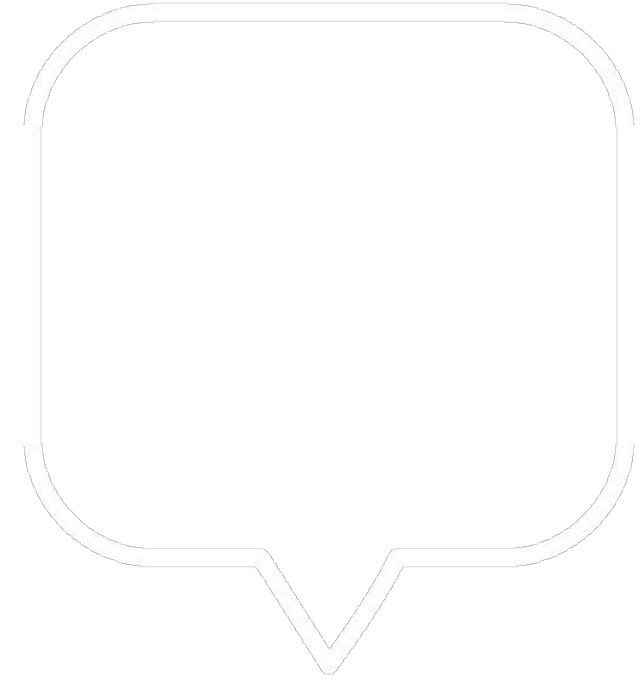
Let's

Combine databases with Flask APIs

Code!



Any Questions?



Quiz Time!

Thanks